UC San Diego

CSE/DSC 234 — Project 1

RAG & Context Engineering + RapidFire AI OSS

Spring 2026

Outline

1. Part 1 (Course project overview – ~14 min + DataHub setup – ~3 min)
2. Part 2 (RapidFire AI OSS ~14 min + RAG with RapidFire AI~5 min)
3. Submission logistics – ~4 min
4. Live Demo – ~10-15 mins

Part I: Course Project Overview

Learning outcomes

- Build an end-to-end RAG pipeline: chunking, embedding, indexing, retrieval, reranking, generation
- Create ≥20 golden Q&A pairs with source locations (data curation)
- Apply retrieval + generation metrics; LLM-as-judge; optional custom metrics
- Use RapidFire AI OSS for systematic multi-config experiments
- Develop judgment on retrieval/context tradeoffs for real applications

High-level task

- Single-turn Q&A over the RapidFire AI OSS documentation corpus
- Use RapidFire's multi-config framework to explore and optimize the pipeline
- Hard constraint: ≤2,000 tokens per query to the LLM (system + all context)
- Models production: corpora are huge vs. context window - retrieval engineering matters

3 main things for you to do

1. Golden data: ≥20 hand-labeled Q&A pairs with identified source locations
2. System: RAG that retrieves relevant chunks and answers within the budget
3. Experimentation: compare multiple configs (chunking, embedding, retrieval, rerank, prompts)

Infrastructure

- Compute: DSMLP DataHub 8-core + 32 GB RAM machine
- LLM APIs: Triton API Gateway
- Framework: pip install rapidfireai; run_evals() API for multi-config RAG evaluation
- RAG: LangChain via RFLangChainRagSpec
- Embeddings: OpenAI or CPU OSS (e.g. sentence-transformers, BGE)

Vector stores & retrievers

- FAISS (default): in-memory, Create mode - quick start
- PGVector / Pinecone: persistent; Read/Update —> embed once, many retrieval configs
- Any LangChain BaseRetriever: BM25, hybrid, ensemble, custom
- Built-in: similarity / score threshold / MMR, top-k —> all experiment knobs
- Optional cross-encoder reranking (e.g. CrossEncoderReranker)

Release packet & evaluation

- Starter setup, doc corpus (RST plaintext), 20 Q&A pairs released val set
- Retrieval script: F1, Precision@k, Recall@k vs. ground-truth spans
- LLM judge (0–5): Correctness, Completeness, Faithfulness
- Hidden test + two hidden judge metrics; leaderboard formula combines retrieval & generation

Leaderboard score (summary)

- Overall = 0.5 × Retrieval + 0.5 × Generation
- Retrieval = average of F1 + Precision@5 + Recall@5
- Generation = average of five 0–5 judge scores (incl. two hidden)
- Leaderboard is for extra credit only (top 10th and 20th percentiles)

Deliverables you must produce

- ≥20 golden Q&A JSON (quality, diversity: factual, conceptual, procedural, comparative)
- main.py: python3 main.py --input … --output … (deterministic final pipeline)
- Output JSON: question_id, answer, sources (tuples per spec)
- Report: pipeline, methodology (≥8 distinct configs), results, AI tools statement
- Private git repo + logs/ (rapidfire.log, experiments_path metrics) with full unaltered commit/push history

Grading snapshot

- Auto-grade main.py: 6% | Golden data: 2% | Report: 2%
- F2F interview: 10% (Pass / Re-interview path)
- Experimentation is assessed via logs, report, interview -> TAs replay run_evals, not every IC op
- Final pipeline must be deterministic; experimentation can be interactive

Datahub & TritonAI LLM Gateway

- Datahub: https://datahub.ucsd.edu/ — sign in with UCSD SSO; launch the 8-core, 32 GiB RAM instance for the project.
- TritonAI LLM Gateway: your API key is in api-key.txt in your user home directory on Datahub.
- Models and endpoints: https://tritonai-api.ucsd.edu/ui/model_hub_table/ — the gateway is OpenAI-compatible.

Timeline and key dates

- Release / TA discussion: Wed, Apr 15
- Set up github repo and add us as collaborators: Thu Apr 16, 11:59pm PT
    - arunkk09
    - hssingh15
    - raghav10j
    - hrb0755
    - manasjain26
- Early submit (leaderboard preview): Thu, Apr 23, 11:59pm PT
- Project 1 Due: Wed, Apr 29, 11:59pm PT
- F2F Interviews: Tue, Apr 28 to Wed, May 6

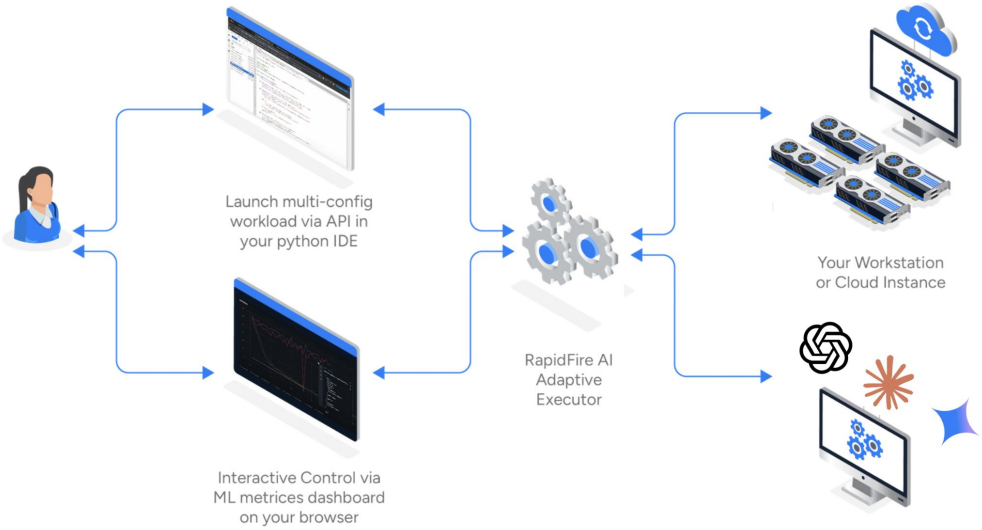
Part II: RapidFire AI OSS

What is RapidFire AI?

- OSS experiment execution framework for LLM pipelines
- Moves from slow sequential tuning to parallel, intelligent workflows
- Hyperparallel execution + dynamic real-time control + automatic backend optimization
- pip install rapidfireai -> CPU-only, single-GPU, or multi-GPU
- Source: https://oss-docs.rapidfire.ai/en/latest/overview.html

Three-way live system

- Your IDE where you launch experiments
- RapidFire Metrics dashboard + control UI
- Multi-core / multi-GPU execution backend
- First-class linkage between editing, observing, and executing runs



How you work with it

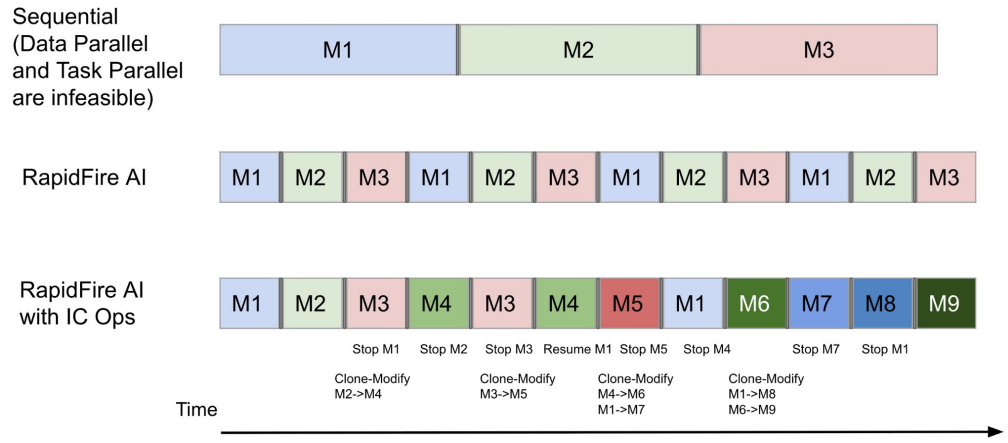
- Start the server and init evals mode from the CLI or notebook cell
- import rapidfireai in notebook or scripts
- Define groups of configs to compare; launch them together via the run_evals() API
- Metrics appear as plots in the ML dashboard; for RAG/context evals, they also appear in the notebook itself as a table
- Interactive Control (IC) Operations: Stop, Resume, Clone-Modify runs in flight

Adaptive execution engine (core idea)

- Training/eval data split into random shards (chunks)
- Schedules all runs on one shard at a time, then cycles — not full pass before any signal
- For self-hosted models: Swap adapters/models across GPU/DRAM with shared-memory caching (can spill to disk)
- For closed model APIs: Swaps API calls across configs and automatically apportions token rate limits
- Swap overhead often <5% of runtime in reported measurements
- Source: https://oss-docs.rapidfire.ai/en/latest/difference.html

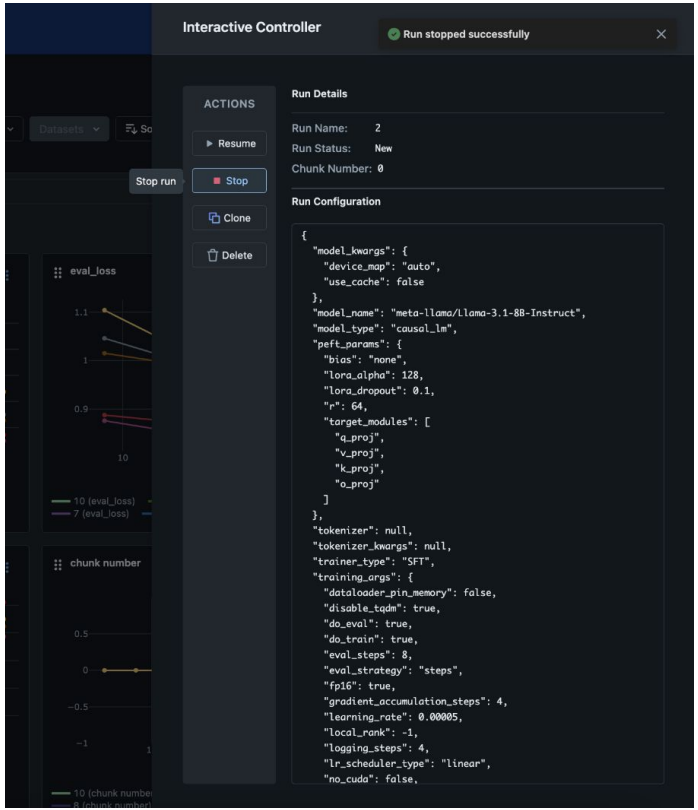
Why sharding vs. sequential

- 1 GPU, multiple configs: see learning behavior on early shards before sequential run 3 starts
- RAG evals: online aggregation — estimated metrics + confidence intervals as shards process
- Converges to exact full-dataset metrics when all shards finish



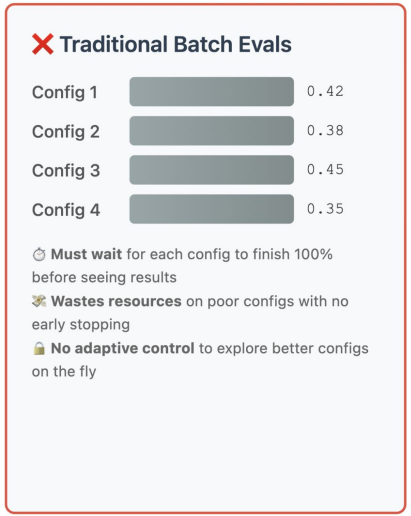
Interactive Control Operations (IC Ops)

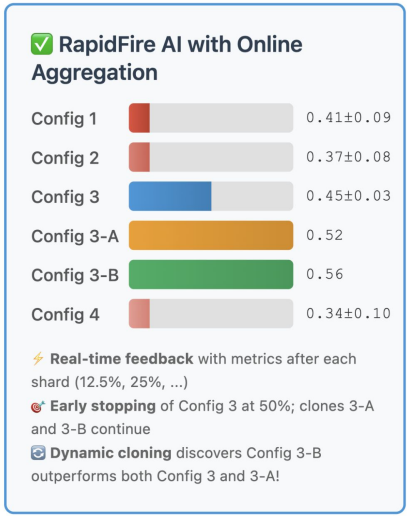
- Stop unpromising runs (wait queue); resume later
- Clone strong runs; modify knobs; warm-start from parent
- Scheduler places runs/chunks on GPUs — focus on experiment logic, not low-level parallelization
- With IC Ops, explore many more configs in similar wall-clock vs. naive sequential



Online aggregation for evals

- Classic batch eval: each config must finish the full eval set before you trust its score -> slow and expensive for LLM calls.
- RapidFire randomly shards the eval set; configs advance in parallel, shard by shard. You get running estimates plus confidence intervals in real time.
- Use that signal with IC Ops: stop weak runs early, clone strong ones, and steer API token spend toward better configs.
- Tag metrics as distributive (additive counts/sums) or algebraic (means/proportions from decomposable parts) so aggregation and CIs are statistically sound.

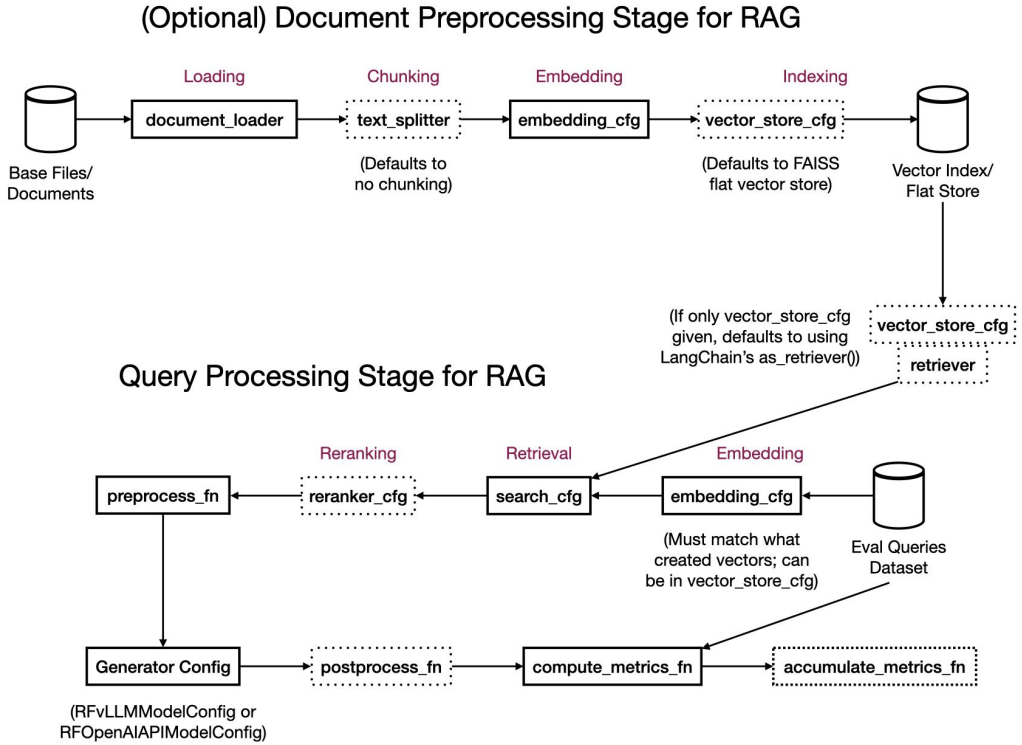




RAG with RapidFire

- You describe one RAG pipeline as a single object: RFLangChainRagSpec (standard LangChain pieces: load, chunk, embed, store, retrieve, optional rerank).
- You attach that object to a generator config: usually RFOpenAIAPIModelConfig for this class (API model + your RAG spec).
- RapidFire runs many such configs in parallel, shows metrics in the notebook, and lets you stop/clone/tweak runs (IC Ops).

RAG with RapidFire



What to do in practice

- Get a baseline working end-to-end first (one chunk size, one embedding, default FAISS, one search top-k).
- Turn knobs into experiments: e.g. several chunk sizes or embedding models by passing List(...) on those fields — that is your multi-config sweep.
- Only open the full API reference when you need a specific feature (PGVector, custom retriever, reranker, few-shot prompts).

Submission logistics

- Submit once per team via Canvas (only the final upload counts if you resubmit).
- Single zipped folder named <TeamID>.zip using your assigned team ID.
- Keep the course Projects page policies in mind (teams, integrity, git rules).
- Git repo expectations
    - Private repo: add all TAs and the instructor as collaborators within 24 hours of release (GitHub IDs on Piazza).
    - Commit and push logs regularly — rapidfire.log and metrics files are required evidence for experimentation and partial credit.
    - History should show real iterative work; dumping AI-generated code in one commit or forging history is an integrity violation.

Evaluation plan

Retrieval evaluation

- Provided script scores overlap between retrieved chunks and ground-truth source spans (file + line ranges).
- A retrieved chunk counts as a hit if it overlaps any ground-truth span for that question.
- Reported retrieval ingredients include F1, Precision@k, and Recall@k for specified k (used in the leaderboard retrieval score).

Generation evaluation

- LLM-as-judge on OpenAI; each metric scored 0–5.
- Released rubric: Correctness, Completeness, Faithfulness (grounded in retrieved context).
- Hidden test adds two additional judge metrics (also 0–5) so teams cannot overfit the public rubric alone.

UC San Diego

Thank you!